

# SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

## ASSESSMENT TOOL 2 – Poster Presentation

|                         |                                                                                                                                     |                      |                                            |
|-------------------------|-------------------------------------------------------------------------------------------------------------------------------------|----------------------|--------------------------------------------|
| <b>Course Code:</b>     | <b>DSA0405</b>                                                                                                                      | <b>Course Title:</b> | <b>FUNDAMENTALS OF DATA SCIENCE</b>        |
| <b>CO Assessed:</b>     | <b>CO3 –Poster Presentation</b>                                                                                                     | <b>Assessment:</b>   | <b>Assessment Tool 2– Coding Challenge</b> |
| <b>Weightage:</b>       | <b>40% of CIA</b>                                                                                                                   | <b>Total Marks:</b>  | <b>100</b>                                 |
| <b>CO5 Statement:</b>   | Analyze and extract image features using detection and matching techniques for effective image representation and comparison. (BL4) |                      |                                            |
| <b>Student Name:</b>    | V.Sai Charan                                                                                                                        | <b>Reg. No.:</b>     | 192472159                                  |
| <b>Section / Batch:</b> | CSE – AI                                                                                                                            | <b>Date:</b>         | 27/08/26                                   |

### Code of Conduct

I V.Sai Charan – 192472159 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: V.Sai Charan

| Criteria                | Weightage | Level 4 – Exemplary                                              | Level 3 – Proficient                           | Level 2 – Developing                    | Level 1 – Beginning                                  | Marks |
|-------------------------|-----------|------------------------------------------------------------------|------------------------------------------------|-----------------------------------------|------------------------------------------------------|-------|
| Understanding of Case   | 25        | Thorough understanding; clearly identifies all key issues        | Good understanding with most issues identified | Basic understanding; some issues missed | Poor understanding; problem not identified correctly |       |
| Application of Concepts | 25        | Effectively applies relevant concepts to analyze the case deeply | Applies appropriate concepts with minor gaps   | Limited application of concepts         | Fails to apply relevant concepts                     |       |

|                 |           |                                                             |                                             |                                          |                                             |  |
|-----------------|-----------|-------------------------------------------------------------|---------------------------------------------|------------------------------------------|---------------------------------------------|--|
| <b>Analysis</b> | <b>20</b> | <b>In-depth analysis with strong reasoning and insights</b> | <b>Good analysis with logical reasoning</b> | <b>Basic analysis with limited depth</b> | <b>Weak or no analysis; lacks reasoning</b> |  |
|-----------------|-----------|-------------------------------------------------------------|---------------------------------------------|------------------------------------------|---------------------------------------------|--|

|                        |           |                                                                                  |                                                      |                                                |                                              |  |
|------------------------|-----------|----------------------------------------------------------------------------------|------------------------------------------------------|------------------------------------------------|----------------------------------------------|--|
| <b>Solution Design</b> | <b>20</b> | <b>Innovative, feasible solutions with strong justification</b>                  | <b>Logical solutions with adequate justification</b> | <b>Basic solutions with weak justification</b> | <b>Incorrect or no solution provided</b>     |  |
| <b>Clarity</b>         | <b>10</b> | <b>C l e a r , w e l l - structure d , a n d e a s y t o u n d e r s t a n d</b> | <b>Organized and understandable presentation</b>     | <b>Limited clarity and structure</b>           | <b>Poor clarity; difficult to understand</b> |  |

## **Q1. Polynomial Regression and Regularization**

### **Aim**

To implement polynomial regression by manually generating polynomial features up to degree 10, demonstrate overfitting as model complexity increases using training and testing errors, and reduce overfitting using Ridge regularization.

### **Problem Statement**

You are given a small dataset containing one independent variable  $X$  and a continuous target variable  $y$ .

### **Develop a Python program that:**

1. Creates or loads the given small dataset.
2. Splits the dataset into training and testing sets.
3. Manually generates polynomial features from degree 1 to 10.
  - \* Do not use PolynomialFeatures from sklearn.preprocessing.
4. For each polynomial degree:
  - \* Train a polynomial regression model.
  - \* Calculate the training Mean Squared Error (MSE).
  - \* Calculate the testing Mean Squared Error (MSE).
5. Plot the training and testing errors against polynomial degree.
6. Demonstrate that increasing the polynomial degree can cause overfitting, where training error decreases while testing error starts increasing.
7. Apply Ridge Regression to the polynomial features.
8. Use the same range of polynomial degrees and calculate the training and testing MSE after regularization.
9. Plot the Ridge training and testing errors.
10. Compare the results before and after regularization and explain how Ridge reduces overfitting.

## Code

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.metrics import mean_squared_error

# -----
# 1. Create a small dataset
# -----

data = {
    "X": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10,
          11, 12, 13, 14, 15, 16, 17, 18, 19, 20],
    "y": [3, 5, 7, 10, 12, 15, 17, 20, 24, 27,
          30, 34, 38, 42, 45, 50, 55, 59, 65, 70]
}

df = pd.DataFrame(data)

X = df[["X"]]
y = df["y"]

# -----
# 2. Split the dataset
# -----

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

# -----
# 3. Function to manually create polynomial features
```

```

# -----

def create_polynomial_features(X, degree):
    X_poly = np.column_stack(
        [X["X"].values ** i for i in range(1, degree + 1)]
    )
    return X_poly

# -----
# 4. Polynomial Regression
# -----

degrees = range(1, 11)

train_errors = []
test_errors = []

print("POLYNOMIAL REGRESSION")
print("-" * 40)
print("Degree\tTrain MSE\tTest MSE")

for degree in degrees:

    # Manually create polynomial features
    X_train_poly = create_polynomial_features(X_train, degree)
    X_test_poly = create_polynomial_features(X_test, degree)

    # Create model
    model = LinearRegression()

    # Train model
    model.fit(X_train_poly, y_train)

    # Predictions
    y_train_pred = model.predict(X_train_poly)
    y_test_pred = model.predict(X_test_poly)

    # Calculate MSE

```

```
train_mse = mean_squared_error(y_train, y_train_pred)
test_mse = mean_squared_error(y_test, y_test_pred)
```

```
train_errors.append(train_mse)
test_errors.append(test_mse)
```

```
print(f'{degree}\t{train_mse:.4f}\t{test_mse:.4f}')
```

```
# -----
# 5. Plot Polynomial Regression Errors
# -----
```

```
plt.figure(figsize=(10, 6))
```

```
plt.plot(
    degrees,
    train_errors,
    marker="o",
    label="Training MSE"
)
```

```
plt.plot(
    degrees,
    test_errors,
    marker="o",
    label="Testing MSE"
)
```

```
plt.xlabel("Polynomial Degree")
plt.ylabel("Mean Squared Error")
plt.title("Polynomial Regression: Training vs Testing Error")
plt.xticks(list(degrees))
plt.legend()
plt.grid(True)

plt.show()
```

```

# -----
# 6. Ridge Regression
# -----

ridge_train_errors = []
ridge_test_errors = []

alpha = 1.0

print("\nRIDGE REGRESSION")
print("-" * 40)
print("Degree\tTrain MSE\tTest MSE")

for degree in degrees:

    # Manually create polynomial features
    X_train_poly = create_polynomial_features(X_train, degree)
    X_test_poly = create_polynomial_features(X_test, degree)

    # Create Ridge model
    ridge_model = Ridge(alpha=alpha)

    # Train Ridge model
    ridge_model.fit(X_train_poly, y_train)

    # Predictions
    y_train_pred = ridge_model.predict(X_train_poly)
    y_test_pred = ridge_model.predict(X_test_poly)

    # Calculate MSE
    train_mse = mean_squared_error(y_train, y_train_pred)
    test_mse = mean_squared_error(y_test, y_test_pred)

    ridge_train_errors.append(train_mse)
    ridge_test_errors.append(test_mse)

    print(f"{degree}\t{train_mse:.4f}\t{test_mse:.4f}")

```

```
# -----  
# 7. Plot Ridge Regression Errors  
# -----  
  
plt.figure(figsize=(10, 6))  
  
plt.plot(  
    degrees,  
    ridge_train_errors,  
    marker="o",  
    label="Ridge Training MSE"  
)  
  
plt.plot(  
    degrees,  
    ridge_test_errors,  
    marker="o",  
    label="Ridge Testing MSE"  
)  
  
plt.xlabel("Polynomial Degree")  
plt.ylabel("Mean Squared Error")  
plt.title("Ridge Regression: Training vs Testing Error")  
plt.xticks(list(degrees))  
plt.legend()  
plt.grid(True)  
  
plt.show()
```



# Output

```
p21.py > ...
1 import numpy as np
2 import pandas as pd
3 import matplotlib.pyplot as plt
4
5 from sklearn.model_selection import train_test_split
6 from sklearn.linear_model import LinearRegression, Ridge
7 from sklearn.metrics import mean_squared_error
8
9
10 # -----
11 # 1. Create a small dataset
12 # -----
13
14 data = {
15     "X": [1, 2, 3, 4, 5, 6, 7, 8, 9, 10,
16          11, 12, 13, 14, 15, 16, 17, 18, 19, 20],
17     "y": [3, 5, 7, 10, 12, 15, 17, 20, 24, 27,
18          30, 34, 38, 42, 45, 50, 55, 59, 65, 70]
19 }
20
21 df = pd.DataFrame(data)
22
23 X = df[["X"]]
24 y = df[["y"]]
25
26
27 # -----
28 # 2. Split the dataset
29 # -----
30
31
32 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=0)
```

Output (⌘+U)

PROBLEMS 27 OUTPUT DEBUG CONSOLE TERMINAL PORTS

saicharan@sais-MacBook-Air FSD LA % clear  
saicharan@sais-MacBook-Air FSD LA % python3 p21.py

POLYNOMIAL REGRESSION

| Degree | Train MSE | Test MSE |
|--------|-----------|----------|
| 1      | 6.0459    | 13.2839  |
| 2      | 0.1278    | 0.3213   |
| 3      | 0.1233    | 0.2117   |
| 4      | 0.1216    | 0.2617   |
| 5      | 0.1193    | 0.2024   |
| 6      | 0.0666    | 12.7215  |
| 7      | 0.0661    | 17.5277  |
| 8      | 0.0661    | 17.6306  |
| 9      | 0.0486    | 556.3744 |



